

Mobile Shop CRUD Project

Features

1. Add Mobile
2. Display All Mobiles
3. Search Mobile
4. Update Mobile
5. Delete Mobile
6. Exit

Data stored in a Python list

Each mobile will be stored as a simple list:

[id, brand, model, price, quantity]

Example:

[101, "Samsung", "Galaxy A55", 35000, 5]

Complete Project

```
#
=====
=====
# MOBILE SHOP CRUD PROJECT
#
=====
=====
# Description:
# This is a simple Mobile Shop CRUD application.
#
# CRUD means:
# C -> Create : Add a new mobile
# R -> Read : Display/Search mobiles
# U -> Update : Update mobile details
# D -> Delete : Delete a mobile
#
=====
=====

# -----
# Global list to store all mobile records
# -----
mobiles = []
```

```

# -----
# Function: add_mobile()
# Purpose:
# Add a new mobile record to the mobiles list.
# -----
def add_mobile():

    print("\n===== ADD MOBILE =====")

    # Take mobile ID from the user
    mobile_id = int(input("Enter Mobile ID: "))

    # Check whether the mobile ID already exists
    for mobile in mobiles:

        if mobile[0] == mobile_id:
            print("Mobile ID already exists.")
            return

    # Take mobile details from the user
    brand = input("Enter Brand: ")
    model = input("Enter Model: ")
    price = float(input("Enter Price: "))
    quantity = int(input("Enter Quantity: "))

    # Create a mobile record using a list
    mobile = [mobile_id, brand, model, price, quantity]

    # Add the mobile record to the main list
    mobiles.append(mobile)

    print("Mobile added successfully.")

# -----
# Function: display_mobiles()
# Purpose:
# Display all mobile records stored in the list.
# -----
def display_mobiles():

```

```

print("\n===== MOBILE LIST =====")

# Check whether the list is empty
if len(mobiles) == 0:
    print("No mobile records available.")
    return

# Display heading
print("-" * 75)

print(
    f"{'ID':<8}"
    f"{'Brand':<15}"
    f"{'Model':<20}"
    f"{'Price':<15}"
    f"{'Quantity':<10}"
)

print("-" * 75)

# Traverse the mobile list
for mobile in mobiles:

    print(
        f"{mobile[0]:<8}"
        f"{mobile[1]:<15}"
        f"{mobile[2]:<20}"
        f"{mobile[3]:<15.2f}"
        f"{mobile[4]:<10}"
    )

print("-" * 75)

# -----
# Function: search_mobile()
# Purpose:
# Search for a mobile using Mobile ID.
# -----
def search_mobile():

```

```

print("\n===== SEARCH MOBILE =====")

# Take Mobile ID from the user
mobile_id = int(input("Enter Mobile ID to search: "))

# Variable to identify whether the mobile was found
found = False

# Search the list
for mobile in mobiles:

    if mobile[0] == mobile_id:

        print("\nMobile Found!")
        print("Mobile ID :", mobile[0])
        print("Brand    :", mobile[1])
        print("Model    :", mobile[2])
        print("Price    :", mobile[3])
        print("Quantity :", mobile[4])

        found = True
        break

# Display message if mobile was not found
if found == False:
    print("Mobile not found.")

# -----
# Function: update_mobile()
# Purpose:
# Update the details of an existing mobile.
# -----
def update_mobile():

    print("\n===== UPDATE MOBILE =====")

    # Ask the user which mobile should be updated
    mobile_id = int(input("Enter Mobile ID to update: "))

```

```
# Search for the mobile
```

```
for mobile in mobiles:
```

```
    if mobile[0] == mobile_id:
```

```
        print("\nMobile Found.")
```

```
        # Display old information
```

```
        print("Current Brand   :", mobile[1])
```

```
        print("Current Model   :", mobile[2])
```

```
        print("Current Price   :", mobile[3])
```

```
        print("Current Quantity :", mobile[4])
```

```
        print("\nEnter New Details")
```

```
        # Take new information
```

```
        new_brand = input("Enter New Brand: ")
```

```
        new_model = input("Enter New Model: ")
```

```
        new_price = float(input("Enter New Price: "))
```

```
        new_quantity = int(input("Enter New Quantity: "))
```

```
        # Update the existing list elements
```

```
        mobile[1] = new_brand
```

```
        mobile[2] = new_model
```

```
        mobile[3] = new_price
```

```
        mobile[4] = new_quantity
```

```
        print("Mobile updated successfully.")
```

```
    return
```

```
# This message is displayed when no matching ID exists
```

```
print("Mobile not found.")
```

```
# -----
```

```
# Function: delete_mobile()
```

```
# Purpose:
```

```
# Delete a mobile record from the list.
```

```
# -----
```

```
def delete_mobile():
```

```

print("\n===== DELETE MOBILE =====")

# Ask the user for Mobile ID
mobile_id = int(input("Enter Mobile ID to delete: "))

# Search for the mobile
for mobile in mobiles:

    if mobile[0] == mobile_id:

        # Display the mobile before deletion
        print("\nMobile Found.")
        print("Brand :", mobile[1])
        print("Model :", mobile[2])

        # Ask for confirmation
        choice = input("Do you want to delete this mobile? (Y/N): ")

        if choice.upper() == "Y":

            # Remove the mobile record from the list
            mobiles.remove(mobile)

            print("Mobile deleted successfully.")

        else:
            print("Delete operation cancelled.")

    return

# Display message if mobile was not found
print("Mobile not found.")

# -----
# Function: dashboard()
# Purpose:
# Display the main menu and control the application.
#
# match-case is used to select the required operation.

```

```
# -----  
def dashboard():  
  
    while True:  
  
        print("\n")  
        print("=" * 45)  
        print("      MOBILE SHOP MANAGEMENT")  
        print("=" * 45)  
  
        print("1. Add Mobile")  
        print("2. Display All Mobiles")  
        print("3. Search Mobile")  
        print("4. Update Mobile")  
        print("5. Delete Mobile")  
        print("6. Exit")  
  
        print("=" * 45)  
  
        # Take menu choice from the user  
        choice = input("Enter your choice: ")  
  
        # Use match-case to process the menu  
        match choice:  
  
            case "1":  
                add_mobile()  
  
            case "2":  
                display_mobiles()  
  
            case "3":  
                search_mobile()  
  
            case "4":  
                update_mobile()  
  
            case "5":  
                delete_mobile()  
  
            case "6":
```

```

        print("\nThank you for using Mobile Shop Management.")
        break

    case _:
        print("Invalid choice. Please try again.")

# -----
# Function: main()
# Purpose:
# Starting point of the application.
# -----
def main():

    # Call the dashboard function
    dashboard()

# -----
# Program execution starts here
# -----
if __name__ == "__main__":
    main()

```

How the data looks internally

Suppose the user adds three mobiles:

```

mobiles = [
    [101, "Samsung", "Galaxy A55", 35000, 5],
    [102, "Apple", "iPhone 15", 65000, 3],
    [103, "OnePlus", "Nord 4", 30000, 7]
]

```

The structure is:

```

      Mobile Record
      |
      -----
      |   |   |   |   |
      ID Brand Model Price Qty

```

For example:

[101, "Samsung", "Galaxy A55", 35000, 5]

means:

101 → **Mobile ID**

Samsung → **Brand**

Galaxy A55 → **Model**

35000 → **Price**

5 → **Quantity**

CRUD Mapping

Operation	Function	List Operation
Create	add_mobile()	append()
Read	display_mobilēs()	for loop
Read/Search	search_mobile()	for + condition
Update	update_mobile()	Modify list elements
Delete	delete_mobile()	remove()